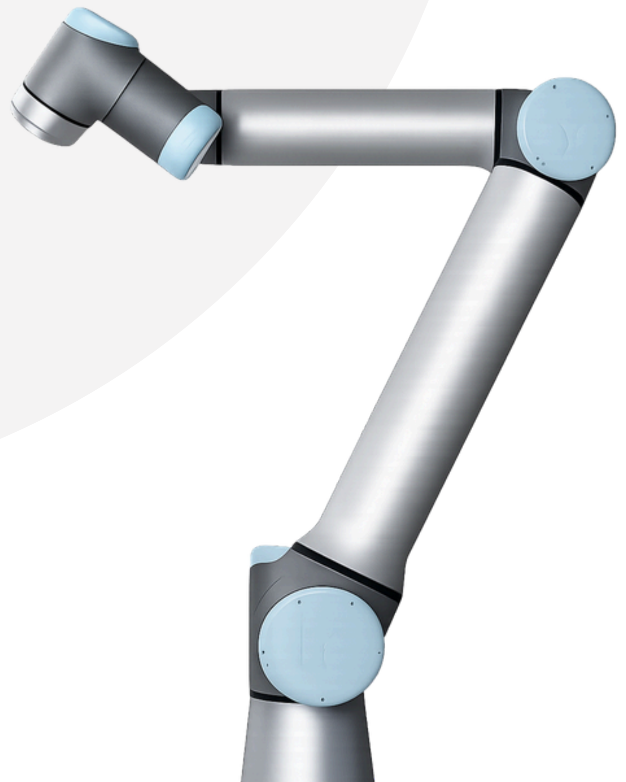
UBC Systopia Lab

# INTEGRATED SIMULATION ENVIRONMENT FOR SDLS - SNAPSHOT

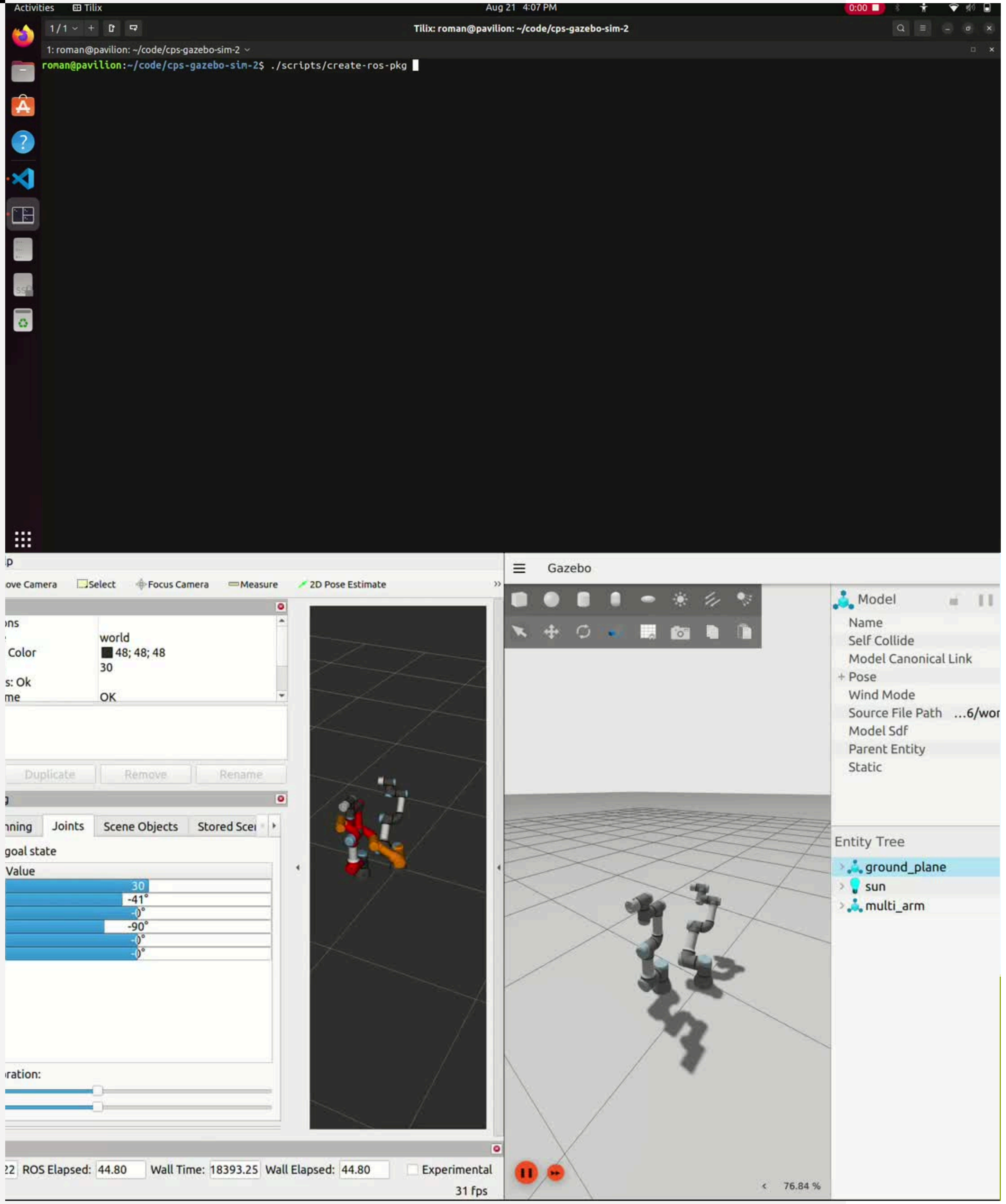
ROMAN SINKUS

22 May, 2026



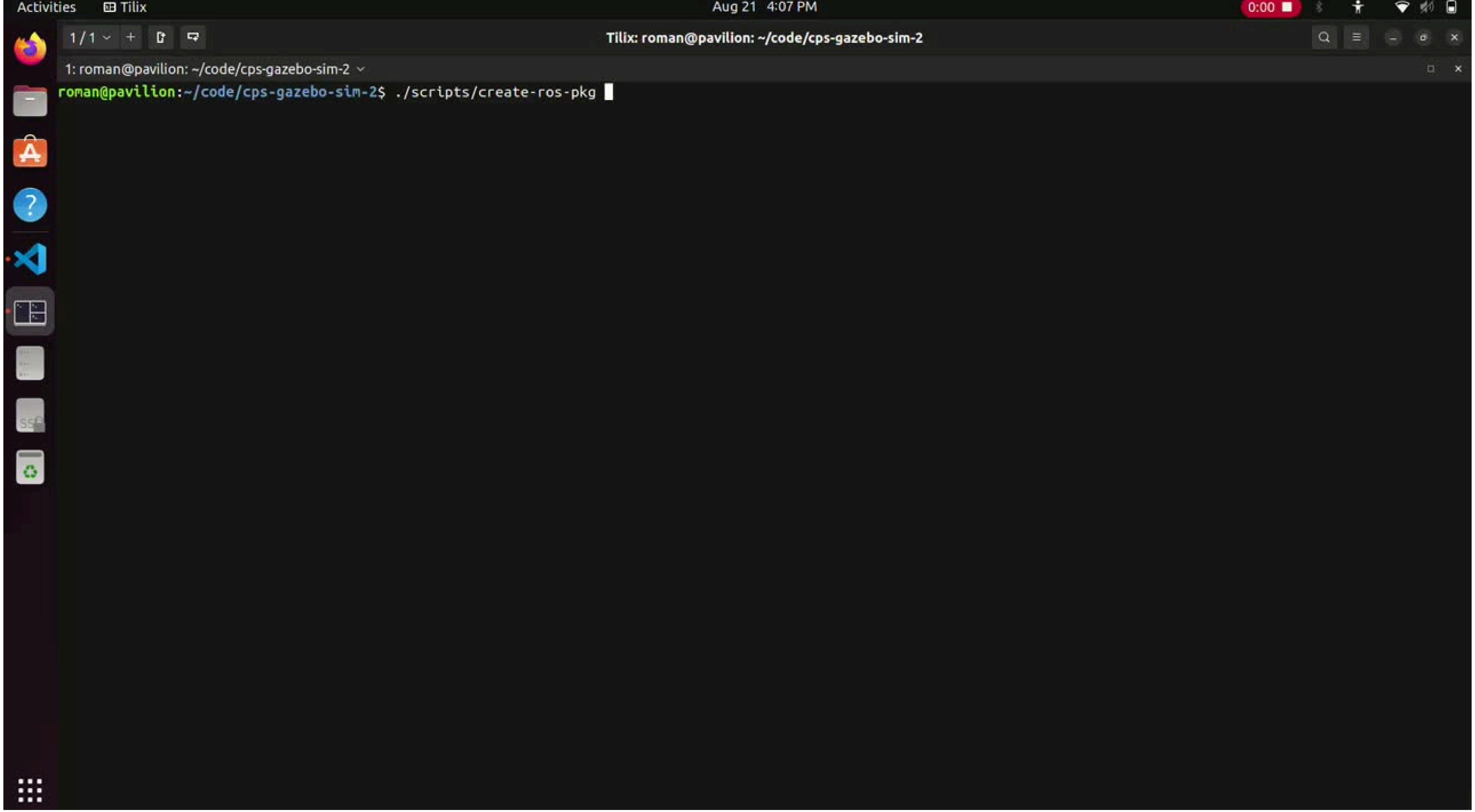
# HIGH-LEVEL

1. Automated ROS package generation overview
2. Varying number of robots
3. Different models of UR robots
4. Integration testing
5. Observations

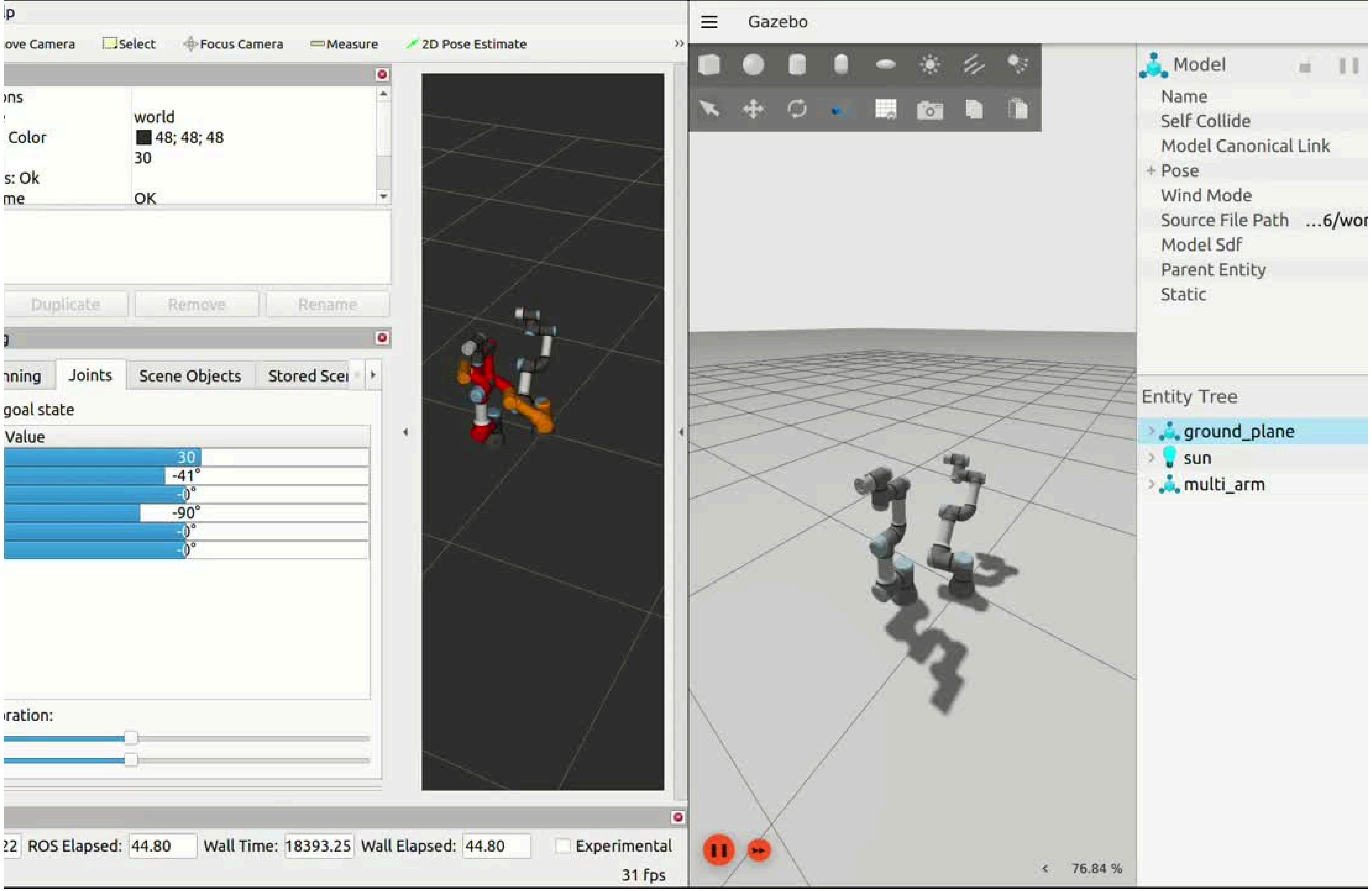



# 1. AUTOMATED ROS PACKAGE GENERATION (OVERVIEW)

- User executes ROS package generation script and supplies inputs to the system via the CLI
  - Number of robots
  - Type of robot
  - Starting x,y,z
  - Starting roll, pitch, yaw
  - Starting Joint positions
- ROS package is automatically generated with all necessary files to bringup the environment
- Configured robots avoid collisions with one another using OMPL (Open Motion Planning Library)

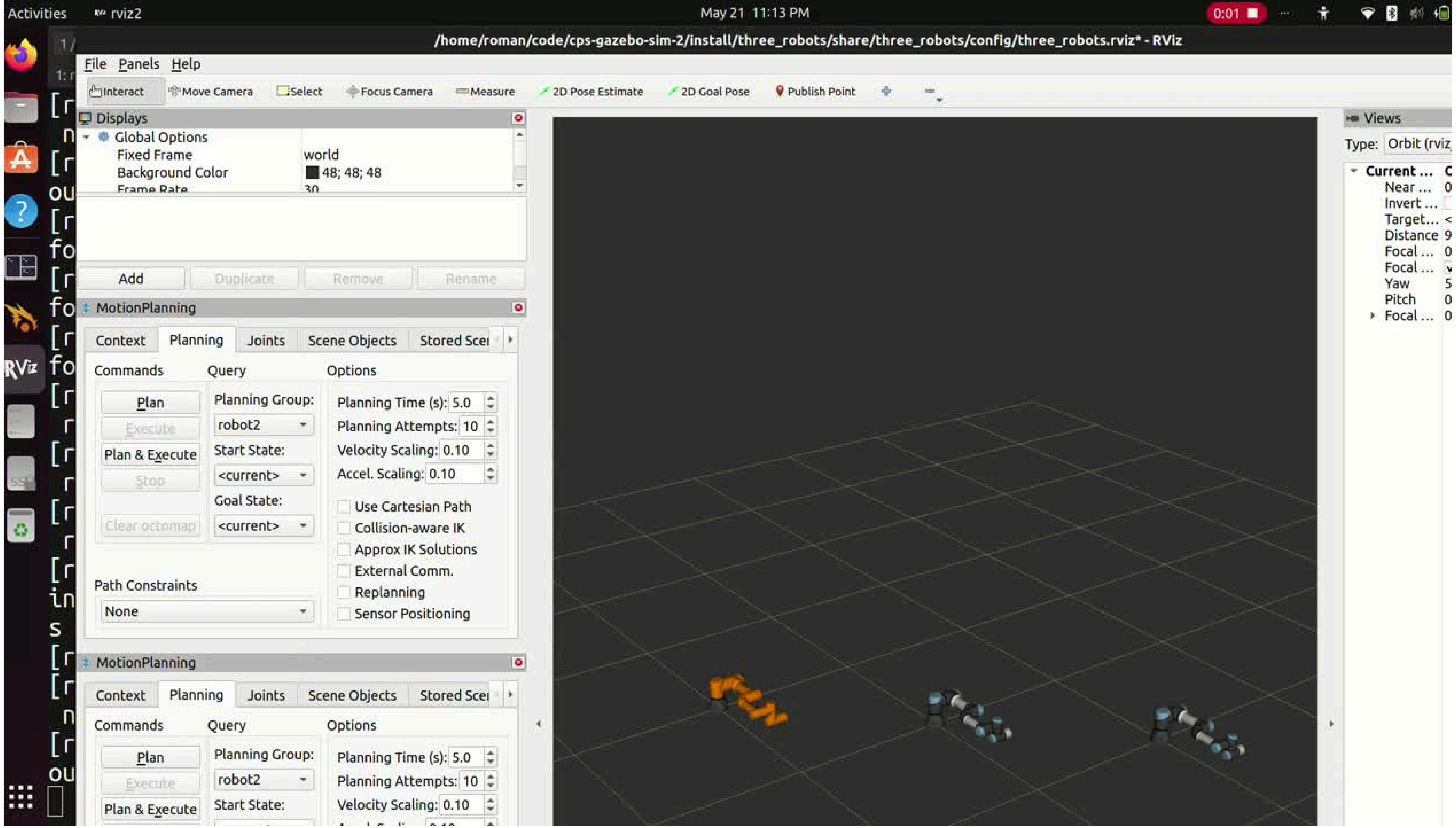



**Video showing the execution of the ROS package generation script and the Gazebo and RViz environments with the newly configured robots.**

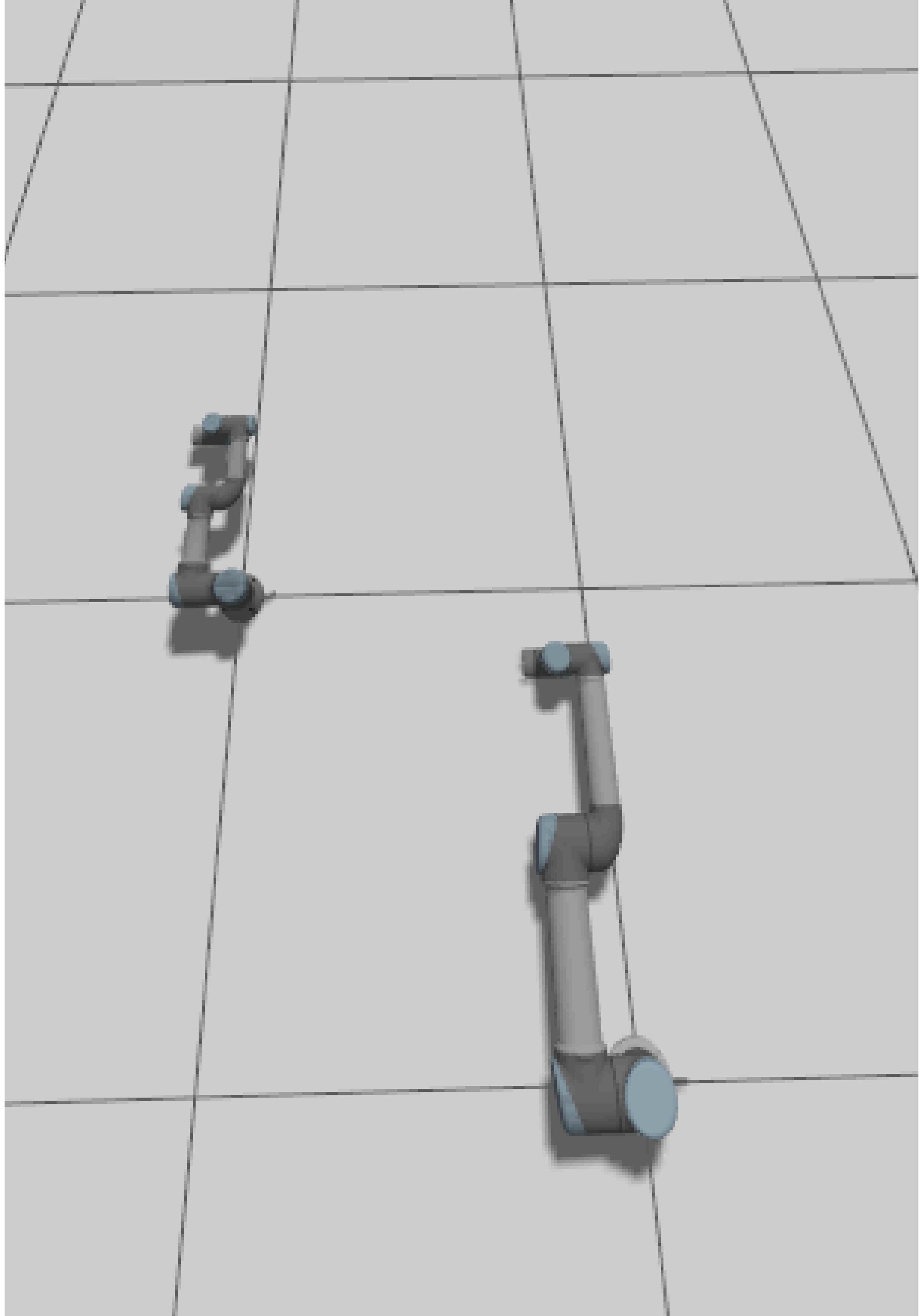



**Video illustrating the UR robot avoiding a collision with the other robot in order to arrive at the goal position.**

# 2. VARYING NUMBER OF ROBOTS (THREE ROBOT EXAMPLE)




**Video illustrating three configured robots created by the ROS package generation script (to show there can be more than two robots generated in the environment).**



# 3. DIFFERENT MODELS OF UR ROBOTS

- Left side: UR3
- Right side: UR5
- Entire list not yet validated

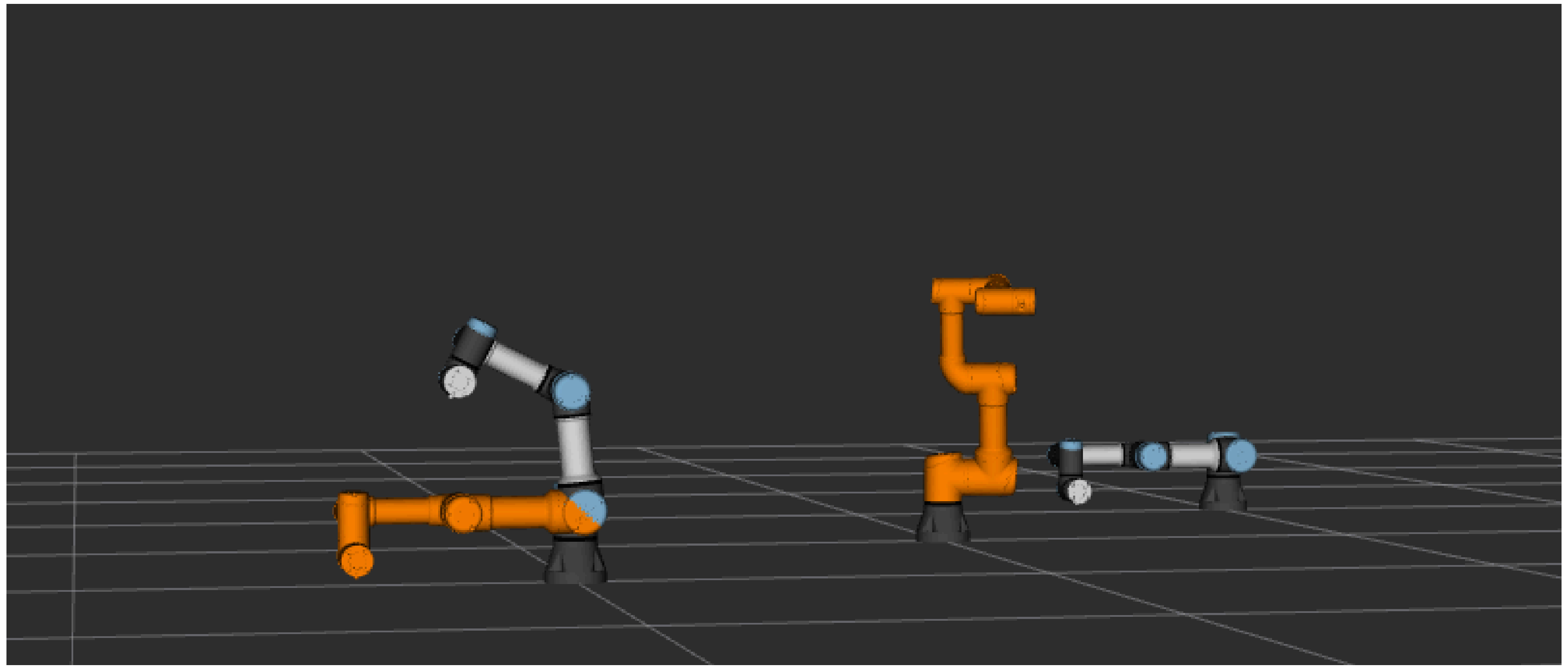
```
[INFO] Available UR robot types for Robot 1:
  1. ur3
  2. ur3e
  3. ur5
  4. ur5e
  5. ur10
  6. ur10e
  7. ur16e
  8. ur20
  9. ur30
```

# 4. INTEGRATION TESTING - COLLISION DETECTON

- Fully headless test
  - Gazebo runs headlessly
  - MoveIt runs without RViz
- Gazebo + MoveIt stack is launched, tests are performed, then teardown kills the process
- Test sends joint states to ROS 2 service which then dictates whether or not there is a collision
  - Collisions are detected by checking whether link collision meshes overlap in 3D space, given the commanded joint configuration
- Static pose check only (path trajectory is not validated)
  - Robots are moved into their goal poses without checking path collisions
  - The test bypasses MoveIt's planning layer entirely and talks directly to ros2_control's joint_trajectory_controller via the FollowJointTrajectory action
- Independent of Gazebo simulation since the collisions are detected using MoveIt

# 5. OBSERVATIONS

- Apparent bugs when using rviz (duplicate robot appearing after moving one of the robots)
  - Not reproducible every time
  - Does not affect Gazebo simulation
- Sometimes the movement of the robot is very slow whereas other times it's much faster
- CLI-interface is useful for walking through the setup but is not user-friendly for editing selections
  - Using a JSON file would allow for regenerating the ROS package with a few edits rather than going through the whole setup each time



# REPO BREAKDOWN

1. .repos
2. docs
3. scripts
4. src
5. test
6. root-level files
7. Observations

# 1. .REPOS

- YAML manifests indicating external repositories for vendors.
- Contains stubs for ned2 and interbotix repos.
- Robotiq grippers yaml file not properly formatted.
- Aim: keep third-party sources out of the repo
- Currently unused infrastructure → must either fix or remove

# 2. DOCS

- Sphinx-based documentation
- Make edits directly to the markdown files found in the docs folder, then run “make html” to rebuild and “_build/html/index.html” to view the documentation
- Functional and currently being used

# 3. SCRIPTS

- The scripts directory contains the user-facing entry points
- Contains the main automation entry point: the “create-ros-pkg” script
  - Uses the scripts in the “helpers/” subdirectory in order to generate all the necessary files while keeping the script modular
- Test script to run the integration test for static collision checks
- Contains convenience launchers for pre-built demos:
  - multiple-robots-moveit
  - multiple-robots-with-controllers-no-moveit
- Environment setup script
  - Still under construction

# 4. SRC

- multi_ur_description/
  - Copy of Universal Robots' official description package containing geometry, joint limits, kinematics, and physical/visual parameters
  - Custom changes have been made to support multiple robots in one Gazebo world
  - Risks associated with having a custom copy of multi_ur_description
    - This design should be re-evaluated for generalizability and up-to-date changes from UR
- Generated ROS packages:
  - All ROS packages generated by the automation scripts
- Legacy content:
  - vendors package
    - currently empty and unused
  - multi_arm_lab_sim_application/
  - multi_arm_lab_sim_bringup/
  - multi_arm_lab_sim_description/
  - multi_arm_lab_sim_gazebo/

# 5. TEST

- Contains current integration test that checks static collisions
- Contains archived tests that are no longer being used
    - The archived tests should be reorganized or removed entirely

# 6. ROOT-LEVEL FILES

- README
- Read the Docs configuration
- Pytest configuration
- Gitignore
- LICENSE
- Outdated, unused and/or broken files:
  - pre-commit configuration
  - setup script
  - requirements and requirements.dev
  - makefile
  - Note: these files should be modified or removed

# 7. OBSERVATIONS

- Currently several of out-of-date or unused files
- Multiple instances of scaffolding without enforcement → broken or unfinished organizational structure

# ACTION ITEMS

1. Snapshot of project
2. Test bringup scripts with multiple robots and multiple types
3. Research other sdls - determine what hardware and software they use

---

4. Observe how SDLs control the robots that don’t use ROS and document this
5. IvoryOS
5. Re-evaluate scope of project and determine concrete next steps
   a. Develop detailed roadmap/plan (similar to what I did last summer)
6. Repo cleanup, documentation for the current system, and documentation for the environment setup
7. Further test the automation bringup scripts and identify any bugs.
   a. Write tests if applicable
   b. Transition the CLI-based interface into a JSON file
8. Add additional collision detection integration tests (if applicable)
9. Add static objects to the environment
10. Add support for other robot vendors